operations. Each of these timers is already being used in one capacity or another, usually for handling hardware PWM and system timing. The `millis()` function makes use of one of these hardware timers to keep a running counter of how many milliseconds the microcontroller has been running since the last time it was turned on or reset. Because this function uses a hardware timer, it does its counting in the background with no impact on the flow or resources of our source code. The `millis()` function should look familiar from Chapter 5, where we used it to count button presses.

There is no additional syntax or parameters for the `millis()` function. By calling the function, it returns a value in milliseconds that can be used like any other variable as part of a conditional test, to perform arithmetic operations on, or assigned to other variables. Because this function returns a value in an unsigned long data type, it will overflow, or reset to 0, in about 50 days. It can also create strange problems if an expression is performed on it using other data types like integers. To make use of the `millis()` function, we might write a line of code like the following:

```
unsigned long startTime = millis();
```

In this example, we declared a variable called `startTime` as an unsigned long data type and assigned the current value of `millis()` to this variable. This way we can keep track of the starting time of some statements or event that might need to be compared to later. Because the delay() function stops all other code from running for the entire time period of the delay, we can keep track of time using `millis()` to avoid using the `delay()` function. In way of example, Listing 7-1 provides an alternative blink example that uses `millis()` instead of the conventional `delay()`.

Listing 7-1. *Blink Without delay()*

```
const int ledPin = 13;

int state = LOW;

unsigned long startTime = 0;
unsigned long interval = 500;

void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  if (startTime + interval < millis()) {
    state = !state;
    digitalWrite(ledPin, state);
    startTime = millis();
  }
}
```

This example code can be combined with other things—like reading switches or sensors, or turning on motors or other actuators, or creating a pattern of LED flashes all operating at different intervals—and because it does not use a `delay()` function, everything else in the sketch will continue to work. In our example, it does this by keeping track of the amount of time that the Arduino interface board has been on and compares this to an interval. If the interval has passed, then it will toggle the state of the LED and write that state to the LED pin—alternating between `HIGH` and `LOW` each time through the compound statement. Let's take a look at the following few lines of the code: